ĐẠI HỌC PHENIKAA

TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA

COURSE: KIẾN TRÚC PHẦN MỀM

PROJECT TOPIC: ONLINE MOVIE STREAMING PLATFORM (NOZIE PLATFORM)

Instructor: Vũ Quang Dũng

Group 3: Nguyễn Duy Minh (23010359 -K17-KTPM)

Role: Frontend/Flutter Developer. Responsible for the mobile client application and user experience (UX).

Class: CSE703110-1-2-25(N02)

Hà Nội, tháng 2 năm 2026

MỤC LỤC

1. Executive Summary

1.1 Overall Project Objectives

The primary objective of the Nozie project is to design and implement a production grade movie streaming platform that demonstrates mastery of key software architecture concepts.

The system is designed to support core workflows—including user registration, catalog browsing, subscription management, and notifications—while satisfying critical architectural drivers such as scalability, availability, and modifiability.

1.2 Main Functional Scope

User Features:

• Authentication: Secure registration and login functionality.

• Catalog Management: Browse, search, and view detailed movie information.

• Subscription Management: Purchase and manage plans (e.g., Free, Premium).

• Streaming: Access and watch content restricted by current subscription levels.

• Notifications: Receive alerts regarding registration, subscription activation, and payments.

Administrative Features:

• Maintain consistent customer profiles and verify subscription statuses across distributed services.

1.3 Key Architectural Decisions

The project adopts a Microservice Architecture pattern, ensuring loose coupling and high cohesion. Key technical decisions include:

• Database-per-Service: Each microservice maintains its own data sovereignty.

• API Gateway: Acts as the single entry point for the Flutter client, routing requests to appropriate backend services.

• Hybrid Communication:

  o Synchronous: REST APIs are used for direct client-to-gateway and gateway-to-service communication.

  o Asynchronous (Event-Driven): RabbitMQ is utilized to decouple services (e.g., Payment, Customer, and Notification), ensuring resilience and scalability.

• Layered Internal Architecture: Each service follows a strict separation of concerns (Controller → Service → Repository).

• Scalable Deployment: The deployment view is modeled to support independent scaling of high-load services, such as Movie and Payment.

1.4 Main Outcomes and Final Demo

The project concludes with a fully functional, end-to-end system. The final demonstration validates that a user can successfully:

1. Register and authenticate.

2. Browse the movie catalog.

3. Select a subscription plan and complete the payment flow.

4. Experience immediate unlocking of premium content via real-time status updates.

2. Project Requirements & Goals

2.1 Problem Background & Business Context

The Nozie Movie Platform addresses the market demand for a flexible and scalable online streaming service that allows users to discover content, subscribe to tiered plans, and stream across multiple devices.

Traditional monolithic architectures often hinder rapid evolution, making it difficult to introduce new capabilities—such as advanced recommendation engines, promotional campaigns, or diverse payment methods—without risking system stability.

Consequently, this project focuses not merely on delivering basic streaming functionality, but on engineering an architecture capable of evolving alongside future business demands (e.g., new subscription tiers, partner integrations, and regional catalogs) while maintaining loose coupling between teams and services.

2.2 Functional Requirements (FRs)

The system is designed to support the following core functional flows:

• FR-01: Registration & Authentication Users must be able to securely register and log in to the system.

• FR-02: Catalog Management Users must be able to browse, search the movie catalog, and view detailed metadata for specific titles.

• FR-03: Subscription & Payment The system must allow users to select subscription plans (e.g., Free, Premium, VIP) and initiate secure payment flows.

• FR-04: Access Control & Enforcement Upon successful payment, the user's subscription status must be updated immediately and strictly enforced when attempting to access content.

• FR-05: System Notifications The system must trigger automated notifications regarding registration, subscription activation, and payment receipts.

2.3 Non-Functional Requirements (NFRs)

The architecture is driven by the following critical quality attributes:

• Scalability: The system must handle traffic spikes—such as concurrent browsing or high-volume subscription requests—by allowing specific services (e.g., Movie, Payment) to scale horizontally.

• Availability & Resilience: The architecture must ensure fault isolation. Failures in non-critical services (e.g., Notification) must not block core business flows like authentication or payment processing; the system must degrade gracefully.

• Performance: Critical user interactions, particularly catalog browsing and checkout completion, must exhibit low latency and high responsiveness.

• Security: All APIs accessible via the API Gateway must be protected by robust Authentication and Authorization mechanisms. Sensitive data, particularly payment information, must be handled according to security best practices.

• Modifiability: The architecture must facilitate the addition of new features—such as new subscription tiers, notification channels, or auxiliary services—with minimal impact on existing codebases.

2.4 Architecturally Significant Requirements (ASRs)

Based on the defined FRs and NFRs, several requirements have directly influenced the architectural topology:

• ASR-1: High Scalability for Read-Heavy Traffic

o Impact: The Movie catalog requires horizontal scaling to handle read heavy loads. This necessitates a dedicated Movie Microservice with its own database and independent deployment units.

• ASR-2: Decoupled Payment Side-Effects

o Impact: Payment completion must not be blocked by downstream processes (e.g., sending emails or updating read models). This drives the adoption of an Event-Driven Architecture using RabbitMQ for asynchronous consistency.

• ASR-3: Independent Evolution of Domains

o Impact: Identity, Customer, Payment, Movie, and Notification domains must evolve independently with separate release cycles. This reinforces the Database-per-Service pattern and the definition of clear service contracts.

• ASR-4: Secure, Single Entry Point

o Impact: All external client traffic must be routed through a centralized API Gateway. This centralizes cross-cutting concerns such as authentication, routing, and logging, simplifying the client-side logic.

2.5 Use Case Model & Actors

The system interaction involves the following primary actors:

• End User (Customer): The primary user who registers, authenticates, browses the catalog, manages subscription plans, executes payments, and consumes streaming content based on their access level.

• Content/Platform Administrator: The internal user responsible for managing the movie catalog metadata and monitoring subscription health (note: while modeled, full implementation of the admin dashboard is secondary to the consumer-facing features).

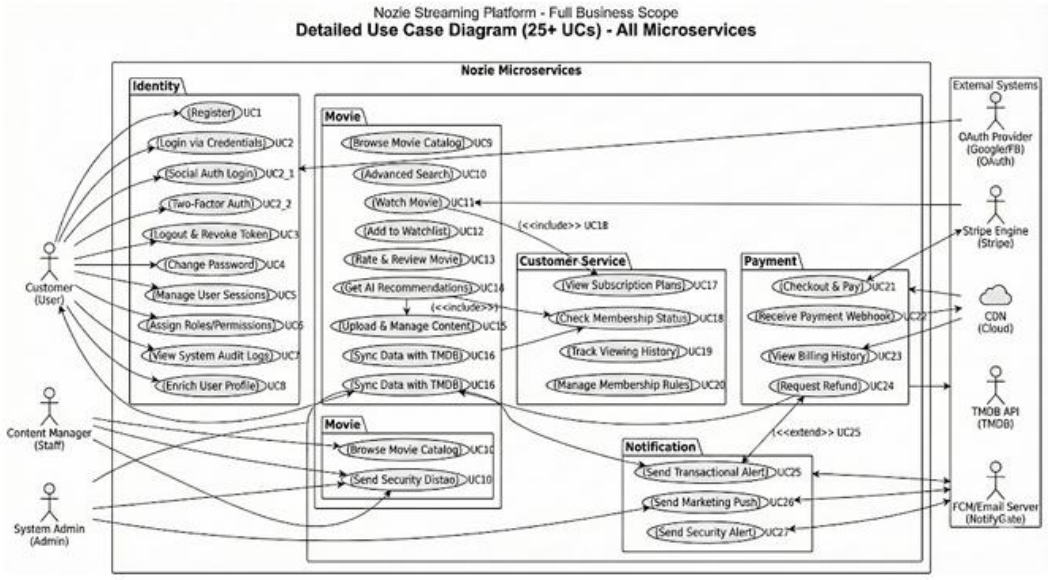


Figure 3.5 Usecase Diagram

Use Case Modeling:

A. Identity Domain (UC1–UC8)

Handles authentication, authorization, and user account management.

Use Case	Actor	Flow Description
UC1 – Register	Customer	1. User submits registration details.2. Identity Service creates account in DB.3. Publishes UserRegisteredEvent.4. Returns success to Client.
UC2 – Login via Credentials	Customer	1. User inputs email/password.2. Identity Service validates hash.3. If valid, issues JWT Access & Refresh Tokens.4. Client stores tokens securely.
UC2_1 – Social Auth Login	Customer	1. User selects “Login with Google/Facebook”.2. Client handles OAuth flow with Provider.3. Identity Service verifies Provider Token.4. Service creates/links account and issues App JWT.
UC2_2 – Two-Factor Auth	Customer	1. After credential check, system requests 2FA code.2. User enters code (SMS/Authenticator).3. Identity Service validates code.4. Issues final Access Token.
UC3 – Logout & Revoke	Customer	1. User selects Logout.2. Client discards local token.3. (Optional) Client calls API to revoke Refresh Token.
UC4 – Change Password	Customer	1. User provides old and new passwords.2. Identity Service validates old password.3. Updates DB with new hash.4. Triggers security notification (UC27).

Use Case	Actor	Flow Description
UC5 – Manage Sessions	Customer	1. User requests active session list.2. Identity Service returns list of devices/IPs.3. User can revoke unknown sessions.
UC6 – Assign Roles	Admin	1. Admin selects user and role (Staff, Manager).2. Identity Service updates user claims.3. Permissions apply on next token refresh.
UC7 – View Audit Logs	Admin	1. Admin requests system logs.2. Identity Service returns login history, failed attempts, and role changes.
UC8 – Enrich Profile	Customer	1. User views/edits profile (Avatar, Name, Bio).2. Client calls PUT /api/users/me.3. Identity Service updates details.

B. Movie Domain (UC9–UC16)

Handles catalog management, streaming, and content discovery.

Use Case	Actor	Flow Description
UC9 – Browse Catalog	Customer	1. Client requests movie list (Latest, Trending).2. Movie Service queries DB with pagination.3. Returns metadata list (Posters, Titles).
UC10 – Advanced Search	Customer	1. User searches by multiple criteria (Actor, Year, Genre).2. Movie Service executes complex query.3. Returns filtered results.
UC11 – Watch Movie	Customer	1. User clicks Play.2. System calls UC18 to verify subscription.3. If allowed, Movie Service returns Stream URL/Manifest.4. Player begins streaming.
UC12 – Add to Watchlist	Customer	1. User adds a title to "My List".2. Movie Service stores userId + movieId association.3. Client updates UI to show "Added".
UC13 – Rate & Review	Customer	1. User rates (1–5 stars) and writes a review.2. Movie Service saves review.3. Updates average rating for the movie.
UC14 – Get AI Recs *(Future/Stub)*	Customer	1. System analyzes user history.2. Calls AI Model to get recommended IDs.3. Movie Service returns personalized list.
UC15 – Upload Content	Staff	1. Staff uploads video file and metadata.2. Movie Service processes file (transcoding) and saves

Use Case	Actor	Flow Description
		details to DB.3. Movie becomes available in catalog.
UC16 – Sync TMDB *(Future/Stub)*	Staff	1. Staff triggers sync.2. Service fetches metadata from TMDB API.3. Updates local DB to match external source.

C. Customer Domain (UC17–UC20)

Handles subscription logic, history, and customer-specific business rules.

Use Case	Actor	Flow Description
UC17 – View Plans	Customer	1. Client requests available subscription tiers.2. Service returns Plan list (Free, Premium, VIP) with features and pricing.
UC18 – Check Status	System	1. Internal/External request to check if User X can access Plan Y.2. Service checks subscriptionStatus and expiryDate.3. Returns TRUE/FALSE.
UC19 – Track History	Customer	1. As user watches (UC11), system logs progress.2. User requests "History".3. Service returns list of recently watched items and timestamps.
UC20 – Manage Rules	Admin	1. Admin configures mapping between Plans and Permissions (e.g., "VIP" = 4K support).2. Service updates business rule configuration.

D. Payment Domain (UC21–UC24)

Handles billing, integration with payment gateways (Stripe), and transactions.

Use Case	Actor	Flow Description
UC21 – Checkout & Pay	Customer	1. User selects plan.2. Payment Service creates intent with Provider (Stripe).3. User completes payment UI.4. Transaction recorded as "Pending".
UC22 – Receive Webhook	System	1. Payment Provider calls Webhook URL.2. Payment Service validates signature.3. Updates transaction to "Success".4. Publishes SubscriptionActivatedEvent (EDA).
UC23 – Billing History	Customer	1. User requests billing logs.2. Payment Service returns list of past invoices/payments (Date, Amount, Plan).

Use Case	Actor	Flow Description
UC24 – Request Refund	Customer	1. User requests refund for eligible transaction.2. System validates policy (e.g., < 7 days).3. Payment Service triggers refund via Provider API.

E. Notification Domain (UC25–UC27)

Handles asynchronous communication via Email/Push.

Use Case	Actor	Flow Description
UC25 – Transactional Alert	System	1. Service consumes RabbitMQ events (e.g., PaymentSucceeded).2. Generates email template.3. Sends confirmation email to user.
UC26 – Marketing Push	Admin	1. Admin drafts a promo message.2. Notification Service queues message for all/selected users.3. Sends Push Notification/Email batch.
UC27 – Security Alert	System	1. Triggered by UC2_2 or UC4.2. Service sends "New Login Detected" or "Password Changed" email to protect user account.

3. Client Implementation

3.1 Client Architecture and Module Organization

The Nozie client is a cross-platform mobile application built using Flutter. It is designed for high performance, smooth animations, and a decoupled architecture that mirrors the backend's microservices structure.

3.1.1 Architectural Pattern: Feature-First Clean Architecture

We adopted a Feature-First organization combined with Clean Architecture principles. Each module is self-contained, consisting of its own Data, Domain, and Presentation layers.

• State Management (Riverpod): Utilizes Riverpod for reactive state management, providing type-safe dependency injection and UI state updates.

• Networking Layer (Dio): API communication is handled by the Dio client with custom Interceptors for JWT injection, global error handling (401 Unauthorized), and development logging.

• Routing (GoRouter): Uses a declarative system with AuthGuards to protect premium content and profile settings.

3.1.2 Design System and Theming

Aesthetic: Follows a Premium Dark Mode theme using deep charcoals, vibrant blues, and gold accents.

• Typography: The Urbanist font family is used for a modern, clean visual identity.

• Localization (slang): Real-time English and Vietnamese support is integrated via the slang package.

Figure 6.1: Folder Structure Screenshot

3.2 Auth & Profile Screens (Identity Management)

This module provides a secure gateway integrating directly with the Identity Microservice.

• Authentication: Features a multi-step validation flow supporting email signup and Google Sign-In.

• JWT Management: Securely persists Access and Refresh tokens, managed by an AuthNotifier to trigger global UI updates.

• User Profile: Includes account settings, password management, notification toggles, and category personalization linked to the Customer Service.

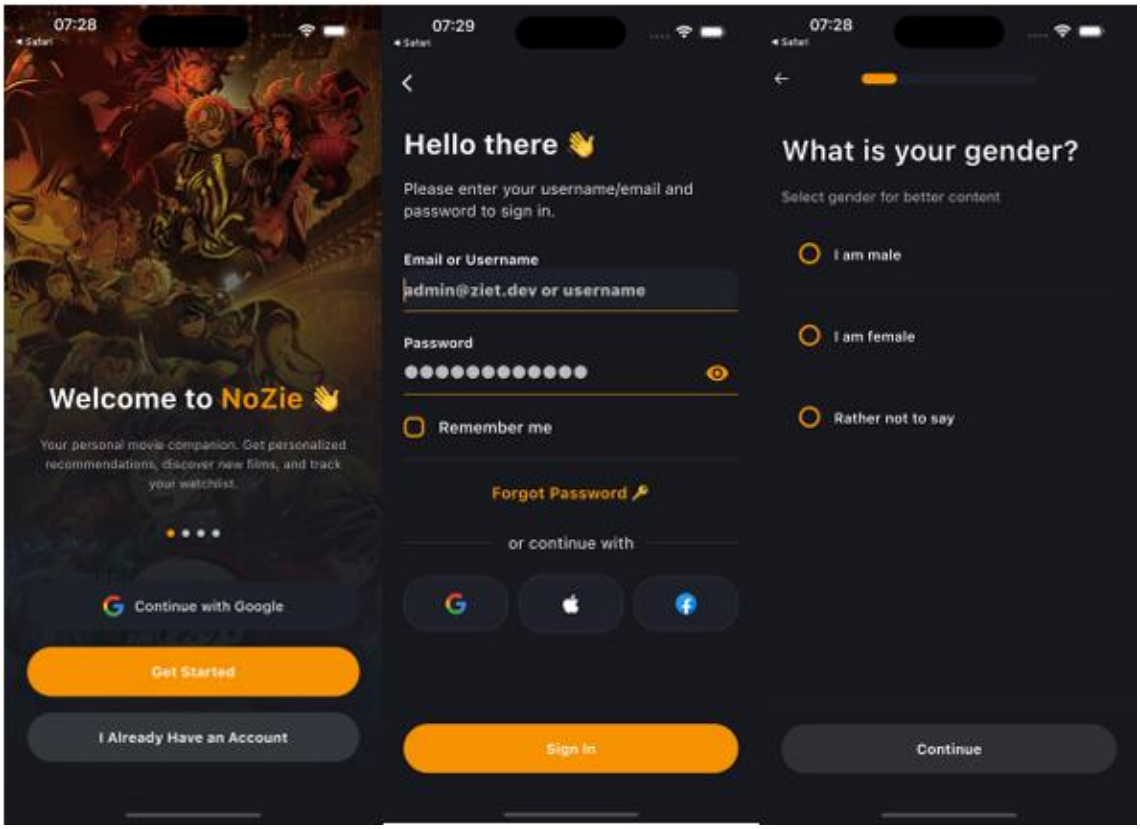


Figure 6.2. Auth & Profile Screens

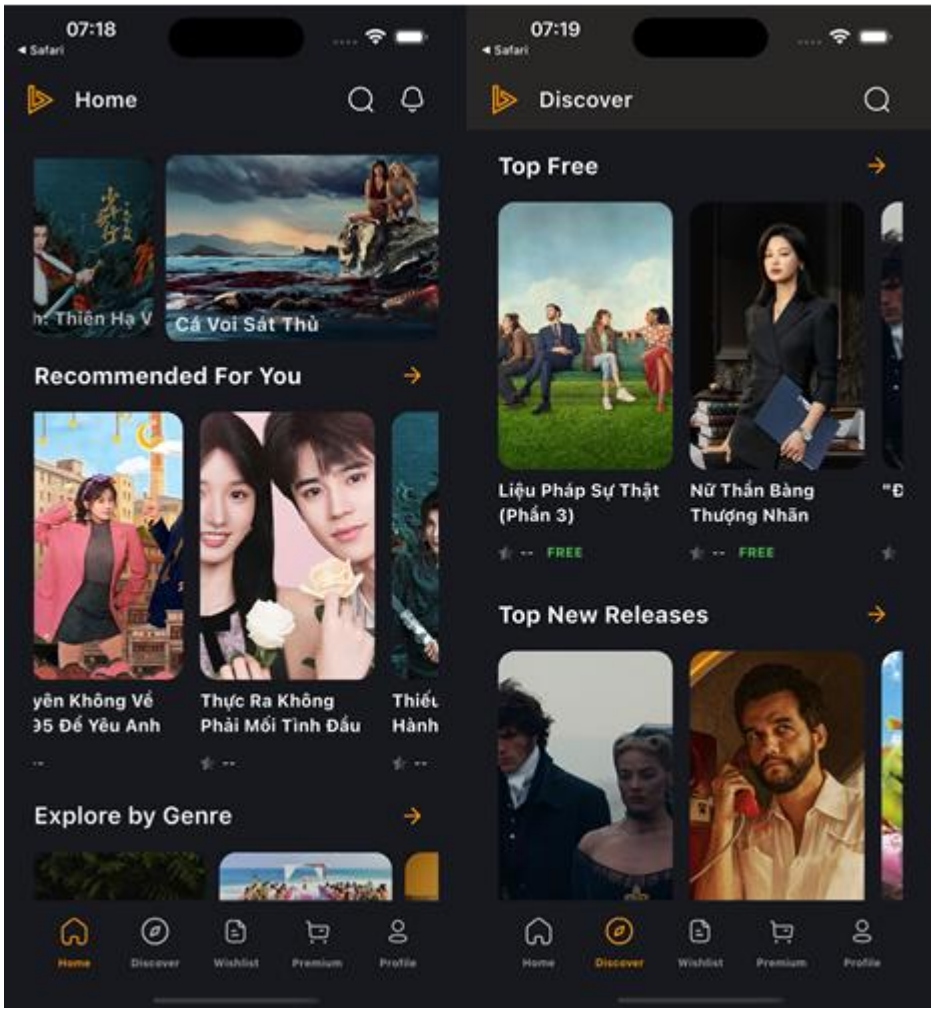
3.3 Movie Catalog & Details Screens (Content Delivery)

The core experience focuses on content discovery and immersive viewing.

• Home Discovery: Features a Hero Section with a smooth-scrolling CarouselView and Dynamic Rows (e.g., "Trending Now"). Skeleton Loading effects are used to improve perceived performance.

• Movie Details: Displays rich metadata from MongoDB, including ratings and synopses.

• Immersive Video Player: Custom UI supporting high-quality MP4/HLS streams with adaptive landscape orientation.


07:18
Safari
Home
Cá Voi Sát Thủ
Recommended For You
Thực Ra Không Phải Mối Tình Đầu
Explore by Genre
Home
Discover
Wishlist
Premium
Profile
07:19
Safari
Discover
Top Free
Liệu Pháp Sự Thật (Phần 3)
FREE
Nữ Thần Bàng Thượng Nhãn
FREE
Top New Releases
Home
Discover
Wishlist
Premium
Profile

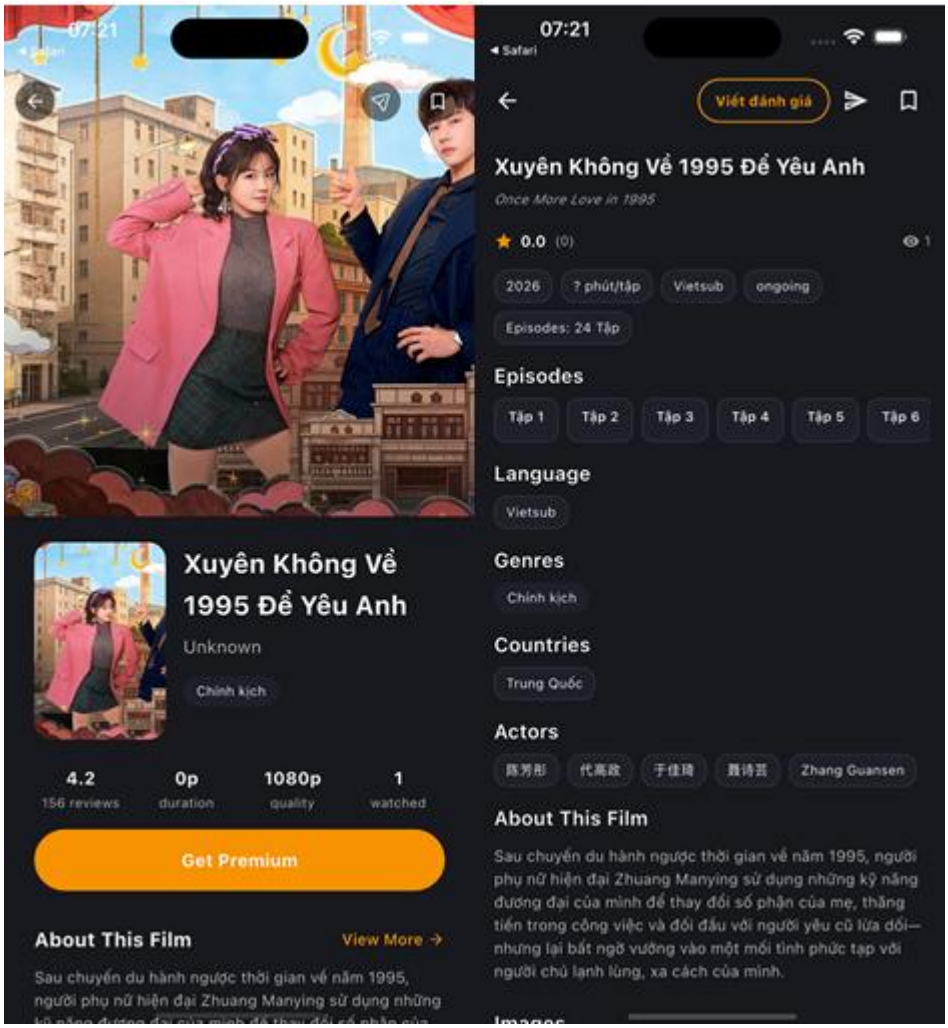


Figure 6.3. Movie Catalog & Details Screens

3.4 Subscription / Payment Screen (Monetization)

Facilitates monetization through a secure, encrypted payment pipeline.

• Plan Selection: A pricing table comparing "Basic" (Free) vs. "Premium" plans.

• Stripe SDK Integration: Utilizes a Native Payment Sheet to ensure PCI DSS Compliance, as the app never touches sensitive card data.

• Real-time Entitlement: Upon transaction confirmation via the backend RabbitMQ flow, the client refreshes the user status to unlock Premium content instantly.

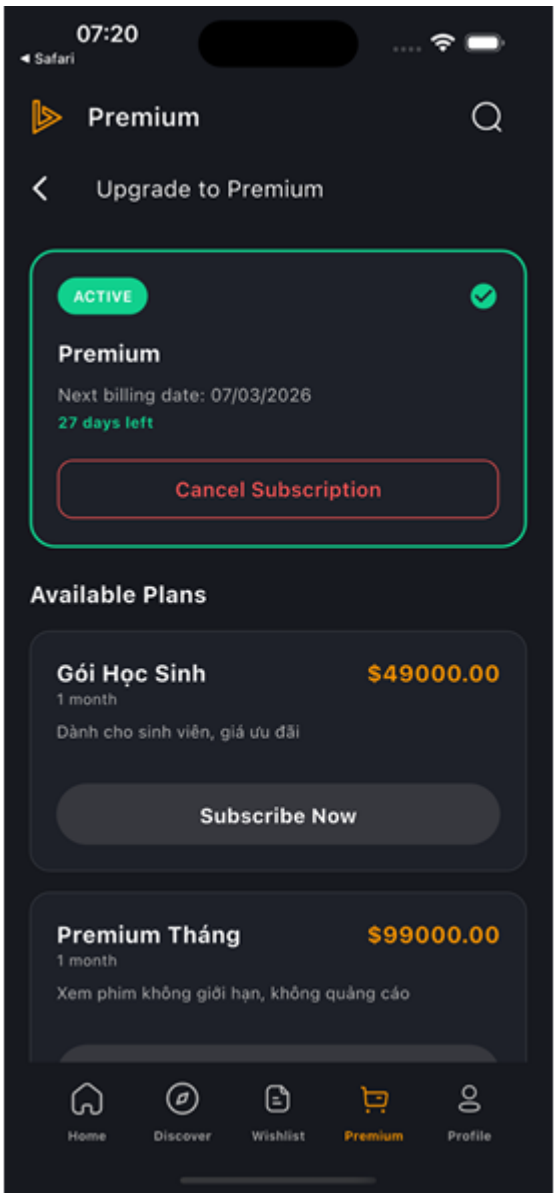


Figure 6.4: Subscription Pricing & Stripe Checkout

4. End-to-End Testing (Flutter ↔ Gateway ↔ Microservices)

This validates the entire integration chain from the mobile client to the database. The "Purchase to Play" Flow Evidence:

1. User Trigger: User clicks "Upgrade to Premium" in the Flutter App.

2. Gateway Traffic: Request passes through AuthenticationFilter and routes to payment-service.

3. Third-Party Integration: Stripe Payment Sheet completes the transaction.

4. Async EDA: payment-service publishes a message to RabbitMQ.

5. Service Sync: notification-service logs: Consuming event: SubscriptionActivated.

6. Final Verification: Flutter App refreshes state; "Locked" movies become accessible.